SIEM LAB · VOLUME 1

Using Your Wazuh SIEM

A beginner's guide to the Wazuh 4.14.8 stack that is already installed and running in your own VirtualBox lab.

Written for	A Linux-only learner building their first SIEM lab
Wazuh version	4.14.8-1 (indexer + manager + dashboard + filebeat)
Runs on	SIEM-Ubuntu — Ubuntu 26.04.1 LTS, 8 GB RAM, 4 vCPU
Lab network	`intnet` isolated segment, 10.0.0.0/24
Already enrolled	Kali Linux as agent `001`
Host	Windows 11 · VirtualBox 7.2.12 (r174389)
Document date	30 September 2026

> **Read this first.** Nothing in this guide installs Wazuh — it is already running and verified. This document is about *operating* it: what each screen means, how to read an alert, how to investigate an event end to end, and how to write your own detection rules. Every command, URL and password is the real one from your lab, so you can follow along literally. Where a value could differ on your machine, it is marked.

Contents

0 How to read this guide

1 What you actually built — the architecture

2 Reaching every part of Wazuh (URLs, ports, credentials)

3 Concepts you need before touching the UI

4 First login and the dashboard tour

5 The Agents page in depth

6 Alerts — the core screen

7 A worked investigation, start to finish

8 Vulnerability Detector

9 SCA — Security Configuration Assessment

10 Compliance dashboards (CIS & PCI DS)

11 File Integrity Monitoring (FIM)

12 Inventory & software inventory

13 Writing your own rules and decoders

14 Manager command-line toolkit

15 Querying Wazuh through the REST API

16 Troubleshooting

17 Maintenance: certs, passwords, backups

18 Lab exercises to work through

19 Where to take this next

0 How to read this guide

You said you know Linux but not security operations. That is actually the better starting position than most people, because a SIEM is just a Linux service collecting logs — the hard part is mental models, not syntax.

Conventions used here

Format	Meaning
`like-this`	A file, folder, service name, index name or field name
`sudo systemctl status wazuh-manager`	Something you type in a terminal. Type it exactly, including the sudo.
blue-tinted output	Output the terminal prints back to you. Compare it to what you get.
low med high crit	Alert severity, matching Wazuh rule levels 0–15.

> **The one habit that will make this whole thing click**
>
> Every time Wazuh shows you an alert, ask three questions: **which machine** (agent name + IP), **what exactly happened** (rule description), and **how do I prove it** (the raw event fields). A SIEM is only useful if you can get from a coloured badge to the underlying evidence without guessing.

What you need before starting

- **SIEM-Ubuntu running.** Wazuh stores its data in memory-heavy Java processes. If the VM is paused or suspended, the dashboard will not answer.
- **A way in.** Either the browser on Windows at `https://127.0.0.1:4433`, or SSH from Windows (`ssh -p 2222 ubuntu@127.0.0.1`), or the VirtualBox console window.
- **Patience on first open.** The dashboard can take 30–60 seconds to answer the first time after a cold boot while OpenSearch finishes loading.

> **TLS certificate warning is expected**
>
> Your dashboard uses a self-signed lab certificate. Your browser will show "Your connection is not private". This is **correct and not a problem** — the certificate was issued by your own lab root CA. Click *Advanced → Proceed to 127.0.0.1 (unsafe)*. Do not do that on any other site.

1 What you actually built — the architecture

Before any UI, you need the picture. Wazuh is not one program. You installed five moving parts, and understanding which part does what is what separates people who can operate a SIEM from people who can only click.

1.1 The five components

Component	Port	Job, in plain English
Wazuh agent	—	A small program installed on every machine you want to watch. It reads local log files, runs security checks, and ships the interesting events to the manager. This is the only part that touches your monitored machines.
Wazuh manager the brain	1514 1515	Runs on SIEM-Ubuntu. Does three things: (1) receives events from agents on 1514, (2) applies thousands of *rules* that decide whether an event is noise or an alert, and (3) runs its own modules — vulnerability scanning, configuration auditing, file integrity monitoring, log collection. Port 1515 is the enrolment port agents use to register themselves.
Wazuh indexer the filing cabinet	9200	A rebranded OpenSearch 7.10.2 cluster. Everything searchable is written here as a document with JSON fields. This is where the dashboard reads from. It is the memory hog — which is why you set the heap to 2 GB in `/etc/default/wazuh-indexer`.
Filebeat the courier	—	Beats version 7.10.2 from Elastic. Its only job in your setup is to read `/var/ossec/logs/alerts/alerts.json` and push it into the indexer. The manager writes its alerts to a file; Filebeat ships that file.
Wazuh dashboard	443	OpenSearch Dashboards, TLS on 443. The web UI. It never talks to the manager directly for alerts — it queries the indexer. It authenticates against the manager API on port 55000 as user `wazuh-wui`.

> **The single most useful sentence in this document**
>
> **Agent → manager → (rules) → alerts.json → Filebeat → indexer → dashboard.** When something does not show up in the UI, you debug along that arrow. If a login event never appears, is the agent alive? If the agent is alive but no alert, is the rule firing? If an alert is in `alerts.json` but not in the UI, it is a Filebeat or indexer problem.

1.2 What is running, right now, on SIEM-Ubuntu

This is the verified state from the install. Confirm it any time with one command:

```
sudo systemctl status wazuh-manager wazuh-indexer wazuh-dashboard filebeat --no-pager
```

Service	Memory	Notes
`wazuh-manager`	–	Core daemons: `wazuh-analysisd`, `wazuh-remoted`, `wazuh-logcollector`, `wazuh-execd`, `wazuh-integratord`, `wazuh-authd`
`wazuh-indexer`	heap 2 GB	Node `node-1`, cluster green, 21 shards
`wazuh-dashboard`	–	TLS on 443, certs owned by user `wazuh-dashboard`
`filebeat`	–	wazuh module 0.1 loaded

1.3 Your lab topology

This is the network you fixed and verified. Keep it in your head — almost every problem in Part 16 of this guide is caused by mixing up these two segments.

Kali 10.0.0.30 → **SIEM-Ubuntu 10.0.0.40** → **WinServer 10.0.0.50**

VirtualBox `intnet` — VM-to-VM only. No gateway, no internet, invisible to Windows.

Separately, each VM has its own NAT adapter on 10.0.3.0/24 for internet. NAT guests *cannot* see each other on that segment — by design.

Machine	Lab IP (intnet)	NAT IP	RAM	Wazuh role
SIEM-Ubuntu	`10.0.0.40`	`10.0.3.15`	8 GB	Server: manager, indexer, dashboard, filebeat
Kali	`10.0.0.30`	`10.0.3.20`	4 GB	Agent `001` — active
WinServer	`10.0.0.50` `planned`	`none yet`	4 GB	Windows Server 2022 — not enrolled yet (see Volume 2)
WindowsClient	`planned`	`none yet`	—	Workstation — not enrolled yet

> **Why agents must reach the manager on the lab IP**
>
> The manager listens on port 1514 on *all* its interfaces, so an agent can connect to `10.0.0.40` (lab net, works) or `10.0.3.15` (NAT, also works for a single guest). But Kali cannot ARP-reach Ubuntu on the NAT segment — that isolation is built into VirtualBox. Your Kali agent is enrolled against `10.0.0.40` for exactly this reason. Always enrol agents using the **intnet** address of SIEM-Ubuntu.

2 Reaching every part of Wazuh

There are four URLs and three credential pairs. Mixing them up is the single most common beginner error, so this is a table worth memorising.

2.1 The URLs

What	From Windows (browser)	Notes
Dashboard (use this one)	`https://127.0.0.1:4433`	Host port forward → VM port 443. This is your main UI.
Dashboard (alternate)	`https://127.0.0.1:443`	**Only from inside the VM** — there is no host forward on 443. Use `ssh -p 2222 ubuntu@127.0.0.1` and open it there.
Indexer API	`https://127.0.0.1:9200`	Raw OpenSearch JSON. Forward is *not* configured — this only works if you are inside the VM, or you add a forward.
Manager API	`https://127.0.0.1:55000`	REST API for agents, rules, SCA. Also not forwarded. See Part 15 for using it via SSH.

Adding a forward so you can reach the indexer from Windows

Worth doing once — it lets you query raw alerts with `curl` from PowerShell, which is invaluable when learning. The VM must be powered off for this command.

```
VBoxManage modifyvm "SIEM-Ubuntu" --natpf2 delete indexer-api
VBoxManage modifyvm "SIEM-Ubuntu" --natpf2 "indexer-api,tcp,127.0.0.1,9200,10.0.3.15,9200"
VBoxManage modifyvm "SIEM-Ubuntu" --natpf2 "manager-api,tcp,127.0.0.1,55000,10.0.3.15,55000"
VBoxManage startvm "SIEM-Ubuntu" --type headless
```

Then from PowerShell. Note `-k` — the certificate is self-signed.

```
curl.exe -k -u admin:<MANAGER_PASSWORD> https://127.0.0.1:9200/_cluster/health?pretty
```

2.2 The credentials

Service	Username	Password
Dashboard login	`admin`	`<MANAGER_PASSWORD>`
Manager API (what the dashboard uses)	`wazuh-wui`	`wazuh-wui`
Indexer API	`admin`	same as dashboard admin
SSH into the VM	`ubuntu`	`ubuntu`
SSH into Kali	`kali`	`kali`

Change all of these before this lab ever touches a real network

These are lab-grade passwords written into a report. The port forwards bind to `127.0.0.1` only, so they are not exposed to your LAN — but a NAT forward still receives connections from anything VirtualBox thinks is local. Treat them as public. Part 17 shows how to rotate the dashboard and API passwords properly.

2.3 SSH access from Windows

From PowerShell, using the forwards that already exist:

```
ssh -p 2222 ubuntu@127.0.0.1     # SIEM-Ubuntu  (works)
ssh -p 2223 kali@127.0.0.1       # Kali        (needs the power-cycle fix, Part 16.4)
```

And from inside SIEM-Ubuntu to Kali, which always works:

```
ssh kali@10.0.0.30
```

Your emergency channel if SSH is broken

Every VM with Guest Additions installed has an out-of-band command channel that needs neither SSH nor working networking. It is how the whole lab was fixed yesterday. Run from Windows:

```
VBoxManage guestcontrol "SIEM-Ubuntu" run --username ubuntu --password ubuntu --exe
"/bin/bash" -- -c "your-command-here"
```

Keep this in your back pocket. It is also why you should never leave a VM suspended instead of shut down when doing network work.

3 Concepts you need before touching the UI

3.1 SIEM, EDR, NDR — what the differences mean

Type	Where it watches	Example
SIEM	Log data from many sources	**Wazuh** — aggregates logs from servers, network gear, apps, cloud
EDR	Endpoint, kernel-level	Microsoft Defender for Endpoint, CrowdStrike
NDR	Network packets, passively	Suricata — *see Part 19, this pairs perfectly with Wazuh*
AV	Endpoint, signature-based	Classic antivirus

Wazuh is primarily a **SIEM**, but its agent also does endpoint work (FIM, SCA, Syscollector), so it is a small EDR too. It is *not* a network sensor — it sees connections because agents report them, not because it sniffs packets.

3.2 Alerts, events, rules, decoders — the four words you will see everywhere

Word	Definition and a Linux analogy
Event	A single log line. `grep` finds events. There are millions.
Decoder	The parser that turns a raw log line into structured JSON fields. analogy a regex or an awk field extractor. Windows Event Log lines like `4624` carry almost nothing usable until decoded.
Rule	A pattern plus a decision. If a decoded event matches, the rule assigns a **level** (severity), a description, and optionally a MITRE ATT&CK tag. analogy a `grep` pattern with an action attached.
Alert	An event that a rule matched and that Wazuh decided was worth a human's attention. If no rule matched, the event is stored but is *not* an alert — this is exactly why an alert's absence does not prove an attack did not happen.

Where the rules live

```
/var/ossec/etc/shared/       # shipped rules + decoders, ~3,000 XML files. Do not edit.
/var/ossec/etc/local_rules.xml # YOUR rules. Read AFTER shared, so you win.
/var/ossec/etc/local_decoder.xml # YOUR custom decoders.
```

3.3 Severity: rule levels 0–15

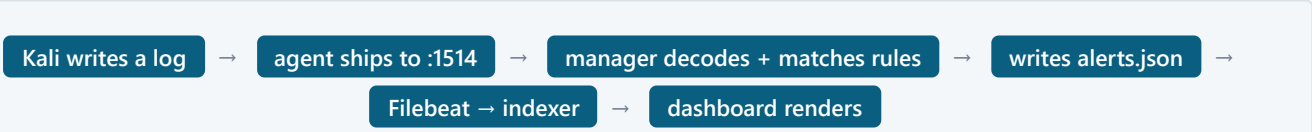
Level	Dashboard tag	Meaning
`0 – 3`	low	Informational. Usually not enabled by default. Not actionable.
`4 – 7`	med	Worth knowing about. Tune or acknowledge.
`8 – 11`	high	Investigate. This is where real incidents live.
`12 – 15`	crit	Respond now.

> **Alert fatigue is the enemy of detection**
>
> If everything is critical, nothing is. A mature workflow is: start at level 8+, work down until you understand the noise, then write *suppression* rules for the boring parts. Part 13 shows how. Never widen the net the other way.

3.4 MITRE ATT&CK

ATT&CK is a public catalogue of attacker tactics, written as `Tactic/Technique` pairs — for example `TA0001/T1078` Valid Accounts, or `TA0002/T1059.001` Execution / PowerShell. Wazuh tags many of its rules with these, which lets you pivot: given an alert, go to the technique and read what a real intrusion using it looks like. You can filter the alerts screen by technique, which is the fastest way to spot a pattern rather than reading alerts one at a time.

3.5 The alert lifecycle in your lab



Debug failures along this arrow in order. It saves hours.

4 First login and the dashboard tour

STEP 1 **Start SIEM-Ubuntu and wait**

Boot it in VirtualBox. The indexer needs 1–2 minutes to form its cluster. Watch it with a loop in the SSH session so you know the moment it is ready:

```
watch -n5 "systemctl is-active wazuh-indexer wazuh-manager wazuh-dashboard filebeat | tr '\n'
' '; echo"
```

Wait until all four print `active`. Only then open the browser.

STEP 2 **Open the dashboard**

Go to `https://127.0.0.1:4433`. Accept the certificate warning. Log in as `admin` with the password from section 2.2.

STEP 3 **Understand the left navigation**

Item	What it is for
Overview	Health summary: alerts by severity, agents online, top talkers. Your landing page.
Agents	Every machine being monitored, its status, its configuration. Part 5.
Alerts	The main event stream, filterable. Part 6 and 7.
Vulnerability	CVE findings per asset from the scanner. Part 8.
System Audit	CIS benchmark results per agent (the SCA module). Part 9.
Compliance	Pre-built CIS and PCI DSS dashboards. Part 10.
FIM	File Integrity Monitoring — who changed which file. Part 11.
Inventory	Hardware, OS, packages, applications. Part 12.
Dev Tools	Browse and edit rules, decoders and SCA policies. Part 13.

STEP 4 **Read the Overview page properly**

Three widgets matter on day one:

- **Alerts by severity** — the coloured bar. Yours will be low-to-medium and dominated by routine sudo/sshd noise. That is normal and healthy.
- **Agents online / never connected / disconnected** — you should see *1 online* (Kali, `001`).
- **Latest alerts** — the live tail. Leave it open; it is the fastest way to confirm an agent is actually shipping.

Your baseline, so you can tell later if something changed

738 alerts were indexed on the install day: **521 from SIEM-Ubuntu, 217 from Kali**. Most are `sudo` commands, SSH sessions, and PAM events. Write this number down. When you start the Volume 2 Windows Server work, "the alert count went from 738 to 4000" will mean something specific, and "it went down" would be worth investigating too.

5 The Agents page in depth

An agent is a monitored machine. This page is how you add them, and it is also your first health check: if a machine is not here and not *active*, it is not sending anything.

5.1 Reading the agent list

Go to **Agents**. You have exactly one row. The columns that will matter to you:

Column	What it tells you
Name	What you typed at enrolment. Use meaningful names — `WINSRV-DC01`, not `DESKTOP-9F3K2`. You will type this name hundreds of times.
Status	Active means it checked in within the keep-alive window. Pending = enrolled, never connected. Never connected = enrolment worked, agent never started. Disconnected = was working, now isn't — the useful one.
Last keep alive	The heartbeat. Default window is 15 minutes. A healthy agent updates every ~10 s, so this reading should be seconds old, not minutes.
Groups	Logical buckets you assign. The main way to scope a search to "my servers".
Labels	Key=value tags for filtering. The cleaner way to handle "which OS is this" or "is this a database box".

5.2 Agent status reference

Status	What happened, and what to do
Active	Working. Do nothing.
Pending	The manager knows about it but the agent has never connected. The enrolment command was never run on the target, or it ran with the wrong manager IP. Fix: run the command.
Disconnected	Worked, then stopped. Check: is the VM powered on? Is it on the same intnet segment? Can it reach port 1514?
Never connected	See Pending, but check the key mismatch too. A different key between enrolment and the running agent config is the usual cause.

5.3 Adding an agent — Linux, the correct way for your lab

> STEP 1 **Open the Add agent form**
> **Agents** → **Add agent**. Choose **Linux**. This page also generates the command; copy it before you leave.

STEP 2 **Fill in the form**

- **Name** — your chosen name, e.g. `ubuntu-server`. Must be unique.
- **IP / CIDR** — `10.0.0.50/24` for a lab VM. Use `any` for machines that roam between networks.
- **Tags** — optional but worth it now: `os.linux,role.server,env.lab`.

STEP 3 **Copy the generated command**

It looks like this. **The key is unique to this agent** — never reuse one, and never paste your manager's key into an agent config by accident:

```
WAZUH_MANAGER='10.0.0.40' WAZUH_AGENT_NAME='ubuntu-server' WAZUH_AGENT_GROUP='servers' \
  curl -s https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.8-
1_amd64.deb \
  | sudo WAZUH_MANAGER='10.0.0.40' WAZUH_AGENT_NAME='ubuntu-server'
WAZUH_AGENT_GROUP='servers' \
    dpkg --install
```

> **Use a lab IP, not a NAT IP**
>
> Fill the manager IP as `10.0.0.40`. If you put `10.0.3.15` it will work from some VMs and silently fail from others, because NAT guests cannot reach each other. This is the mistake you already paid for once with the duplicate 10.0.3.15 address.

STEP 4 **Run it on the target and start the agent**

```
sudo systemctl daemon-reload
sudo systemctl enable --now wazuh-agent
sudo systemctl status wazuh-agent # expect: active (running)
```

STEP 5 **Verify it actually reports**

```
grep "Successfully added agent" /var/log/wazuh-agent/wazuh-agent.log | tail -3
sudo /var/ossec/bin/agent_control -l # on the manager, lists live agents
```

Then refresh the Agents page in the browser. Status should be `Active` within about 30 seconds.

5.4 Enrolling a Windows machine

The UI picks this up automatically — choose **Windows** on the Add agent page and you get a PowerShell one-liner instead of a Debian package. It is covered in full, with all three network options, in **Volume 2**. The short version:

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.8-1.msi `
  -OutFile $env:USERPROFILE\Downloads\wazuh-agent.msi
msiexec.exe /i $env:USERPROFILE\Downloads\wazuh-agent.msi /q `
  WAZUH_MANAGER="10.0.0.40" WAZUH_AGENT_NAME="winsrv-01" WAZUH_AGENT_GROUP="windows,servers"
```

> **Groups versus labels — pick one and be consistent**
>
> **Groups** are enforced on the manager: an agent's configuration is assembled from the groups it belongs to. Use them for "these all get this hardening". **Labels** are free-form metadata for filtering. Use them for "this is a database". Mixing the two up is a very common and very confusing beginner bug.

5.5 Agent management, in one table

Task	How
See a live heartbeat	Agents → your agent → the status dot, or `agent_control -l`
Force a config re-read	`sudo /var/ossec/bin/agent_control -R <ID>` then `-i <ID>`
Delete an agent	Agents → row → … → Delete. Removes its history.
Bulk-import many	Agents → *Add agent* → **Using a file** tab. Fastest path when you add 10 Windows hosts.
Restart the agent remotely	Agent → *Restart* in the UI, or REST `PUT /agents/{id}/restart`
Reset a forgotten key	Agents → *Reset key*. Reinstall on the target with the new key.

6 Alerts — the core screen

This is where you spend your working life. Ninety percent of SIEM skill is getting good at this one screen.

6.1 The default view, column by column

Column	Read it like this
Timestamp	When Wazuh processed it, in the dashboard's timezone. Set your timezone deliberately in Settings → General, or you will misread incidents later.
Agent	Which machine. Filter on this constantly.
Location	Data source — `/var/log/auth.log`, `syscheck`, `sca`, `vuln-detector`, `win_eventchannel`. Tells you which module raised it.
Rule description	Plain-English summary. This is the decoded meaning, not the raw log.
Level	0–15 as a badge. Sort by it.
Rule ID	The ID of the rule that fired. Memorise the handful you see daily. Searchable in Dev Tools.
MITRE	Tactic/technique tags, when the rule has them. Click to pivot.

6.2 Filtering — the actual skill

Use the search bar across the top. It accepts Lucene syntax, which is far more powerful than it looks. Click **Add filter** to build conditions with dropdowns if you prefer.

Query	What it returns
`rule.level:>=10`	High and critical only. Your daily triage starting point.
`agent.name:"kali-linux-2026.2-virtualbox-amd64"`	Everything from Kali alone.
`rule.level:>=8 AND agent.name:"winsrv-01"`	High alerts on one machine. **AND** must be uppercase.
`rule.groups:authentication_failed`	Failed logons, by Wazuh's own taxonomy rather than raw IDs.
`rule.id:5712`	Every alert from one specific rule.
`data.win.system.eventID:4625`	Raw Windows event ID. Your main tool once Windows is enrolled.
`rule.mitre.id:T1078`	Anything ATT&CK-tagged as Valid Accounts.
`location:"vuln-detector"`	Findings from the vulnerability scanner only.
`NOT rule.level:<8 AND agent.name:"kali"`	Suppress the low-level noise to see what is left.
`timestamp:[now-24h TO now]`	Last 24 hours.

> **Save your searches**
>
> Once a query earns its keep, click the bookmark icon and give it a real name like `kali-high-critical`. Saved searches appear in the left rail. This is the difference between a SIEM you use and a SIEM you admire. Build 4–6 of them in your first week and they become your whole workflow.

6.3 Reading one alert in detail

Click any alert row. It expands into three tabs. The one that matters most is **Full event**, which is the decoded JSON. A realistic Linux example:

```
{
  "timestamp": "2026-09-30T01:44:12.330+0000",
  "agent": { "name": "kali-linux-2026.2-virtualbox-amd64", "ip": "10.0.0.30" },
  "rule": {
    "level": 5, "id": "5712",
    "description": "sshd: Authentication failed.",
    "groups": ["sshd", "authentication_failed"],
    "mitre": [{ "technique": "T1110", "tactic": "Credential Access" }]
  },
  "data": {
    "srcip": "10.0.0.40",
    "srcuser": "ubuntu",
    "id": "21001",
    "dstuser": "root"
  },
  "location": "/var/log/auth.log",
  "decoder": "sshd",
  "predecoder": { "program_name": "sshd" }
}
```

Field	How to use it
`rule.id`	Your search key. Look it up in Dev Tools → Rules.
`rule.level`	Severity. Filter on it first, every time.
`data.srcip`	Source IP. On your lab, `10.0.0.x` means another VM; `127.0.0.1` or absent means local.
`data.srcuser`	Account involved. `root` appearing at all is worth a look.
`data.id`	Wazuh's internal decoder ID. Groups related events into one family.
`decoder`	Which parser matched. Tells you which rule family to look at.

7 A worked investigation, start to finish

Reading alerts is passive. Investigating is the actual job. Here is a complete example on your own lab, using only things you have.

1 Notice

On the Overview page the alert count is climbing and you keep seeing medium badges in the latest-alerts feed. Nothing critical.

The lesson: a low-level alert nobody reads is how real intrusions get missed.

2 Triage — filter down, do not read everything

```
rule.level:>=8 AND agent.name:"kali-linux-2026.2-virtualbox-amd64"
```

Empty. Good — so the problem is not something urgent. Now drop the level:

```
rule.level:>=5 AND rule.groups:authentication_failed AND agent.name:"kali-linux-2026.2-virtualbox-amd64"
```

Now you have 40-odd rows instead of 400.

3 Group by source IP

Look at the `data.srcip` column across those rows. One value dominates. In your case it will be `10.0.0.40` — SIEM-Ubuntu. Now you have a hypothesis to test, not a vague worry.

4 Form and test the hypothesis

Hypothesis: something on SIEM-Ubuntu is repeatedly attempting to authenticate to Kali and failing.

Test it against the source rather than the destination. Switch to SIEM-Ubuntu, open a `sudo` shell, and look at what is actually trying:

```
sudo journalctl -u ssh --since "1 hour ago" | grep -i "failed\|invalid" | tail -20
sudo grep "Accepted\|Failed" /var/log/auth.log | awk '{print $NF}' | sort | uniq -c | sort -rn | head
```

5 Find the benign explanation

Almost certainly one of these, all of them yours:

- An `authorized_keys` entry or a cron job on Ubuntu running `ssh user@10.0.0.30` before the key/agent was fully set up.
- NetworkManager or a monitoring probe polling port 22.
- Your own repeated manual `ssh` attempts while debugging the 2223 forward.

Or, if the source is genuinely unfamiliar, keep going. The point is that you now have evidence, and either a boring answer or a real finding.

6 Decide and act

Finding	Action
Benign, expected	Write a **suppression rule** so it stops wasting your attention (Part 13.5). Document it.
Benign but should not exist	Fix the source. A stale cron job is a finding in its own right.
Unknown source, repeated failures	Treat as suspicious. Isolate the VM, capture evidence, note the timeline. This is where a real SOC would escalate.

7 Write it down

Four lines is enough: **what** you saw, **when** (first and last seen), **which host**, **what you concluded and why**. Do this even for the boring ones. Security work that is not written down did not happen, and it is also the raw material for the portfolio writeup in Part 19.

8 Vulnerability Detector

This module scans your managed assets against a feed of known CVEs and tells you which are actually present. It is a scanner, not an IDS — different job, complementary mindset.

8.1 How it works

CVE feed → **agent reports OS + packages** → manager matches versions → **findings in dashboard**

Stock feeds include NVD, Ubuntu, RedHat, Amazon, Debian, Microsoft and Oracle. Some require an API key; the ones that do not work out of the box on your install.

8.2 Using it

View	Question it answers
Vulnerability → Summary	Totals by severity across the fleet. Start here.
Vulnerabilities	Every individual CVE finding, sortable and filterable by agent, package and severity.
Affected agents	Which machines have findings — your patch priority list.
Packages	Inventory of software across all agents.
Inventory	OS versions, so you can spot end-of-life systems.

Filter for your own findings:

```
location:"vuln-detector" AND rule.level:>=7
```

8.3 Turning off false positives — the part everyone skips

A fresh Ubuntu or Windows Server will light up with findings for packages that are not actually vulnerable in a way that matters. Suppressing at the source beats arguing with the dashboard.

STEP 1 **Find the OS detection block**

```
sudo nano /var/ossec/etc/ossec.conf
```

In 4.14 the vuln-detector is configured with `<ossec_config>` inside a `vulnerability-detector` block rather than the old top-level `<vulnerability-detector>` section. Look for it near the bottom.

STEP 2 **Add an ignore entry, then restart**

```
<vulnerability-detector>
  <ignore>
    <os>CVE-2023-0000, CVE-2024-1111</os>
    <package>/usr/share/doc</package>
  </ignore>
</vulnerability-detector>
```

```
sudo /var/ossec/bin/wazuh-logtest -t   # validate XML BEFORE restarting
sudo systemctl restart wazuh-manager
```

> **Never ignore a CVE you have not actually assessed**
>
> An ignore entry is a claim that the finding does not apply. Record why. A suppression list nobody can justify six months later is worse than noise.

9 SCA — Security Configuration Assessment

This is arguably the most valuable module you have. It checks whether your machines follow a hardening standard (CIS benchmarks) and tells you exactly which settings are wrong. Misconfiguration causes more real breaches than any exploit.

9.1 What it is

Wazuh ships policy files that check hundreds of individual settings and grade each `pass`, `fail` or `not applicable`. Policies available on your install:

- **CIS Ubuntu 20.04 / 22.04** — applies to your SIEM-Ubuntu and Kali agents
- **CIS Windows Server 2019 / 2022** — the one you will use in Volume 2
- **GPOs** — a bundled set

9.2 Reading an SCA result

Open **System Audit** → pick your agent → **CIS** tab. You get a long table of checks. Each row has:

Column	Meaning
Title	The setting, e.g. "Ensure password length minimum is 14 or greater".
Result	Pass / Fail / Not applicable.
Remediation	The exact command or GPO change. Follow it literally.
Description	Why the setting matters. Read this before changing anything.

> **This is the module that will make your lab look professional**
>
> A screenshot of a CIS Windows Server 2022 baseline going from mostly-red to mostly-green, because you fixed it yourself, is a genuinely strong portfolio artefact. It demonstrates you understand *why* the settings exist, not just that you can run tools. Volume 2 is built around producing exactly that.

9.3 Running a policy by hand

```
sudo /var/ossec/bin/agent_control -R <AGENT_ID>   # ask agent to re-read config
sudo /var/ossec/bin/agent_control -i <AGENT_ID>   # restart the agent
```

Results appear in the dashboard within a minute. To force a fresh SCA run, restart the agent — the policy is evaluated on agent start.

9.4 Reading the raw SCA log

```
sudo tail -40 /var/ossec/logs/ossec.log | grep -i sca
sudo grep "sca:" /var/ossec/logs/alerts/alerts.json | tail -3 | python3 -m json.tool
```

10 Compliance dashboards

Wazuh ships ready-made dashboards that map SCA results to a named compliance framework, so you can show a manager "we meet CIS Level 1" without producing 400 screenshots.

- **CIS Microsoft Windows Benchmarks** — maps to the Windows CIS benchmarks
- **PCI DSS** — maps findings to PCI DSS requirement 11 (testing) and others

Open **Compliance**, pick a framework, set your date range, and it renders requirement-by-requirement pass rates. Expect to be *far* from 100% on a default Windows Server install — that is the point. The gap between the default state and a hardened state is your project.

> **Compliance dashboards are for reporting, not for finding problems**
>
> Use **System Audit** to find and fix individual failures. Use **Compliance** to show the state of the whole estate to someone who does not want detail. Using the wrong one is how people spend a day fixing nothing.

11 File Integrity Monitoring (FIM)

FIM takes a fingerprint of files, then tells you when a fingerprint changes. It is the module that catches an attacker who got in and did not stay quiet.

11.1 Why it matters

An intruder with remote shell access usually has no malware to detect. They edit `/etc/passwd`, drop a script, or change a config. Every one of those is a FIM event. FIM also detects ransomware and log tampering — if someone clears `/var/log/auth.log`, that is a change to a monitored file.

11.2 How Wazuh detects it

Change type	Meaning
`modified`	Contents changed
`added`	New file appeared in a watched directory
`deleted`	File removed
`hash_changed`	Checksum differs from the stored baseline
`attributes_changed`	Permissions or timestamps changed

11.3 Configuring syscheck

The block is called `syscheck` in `/var/ossec/etc/ossec.conf`.

```
<syscheck>
  <frequency>43200</frequency>
  <scan_on_start>
    <enabled>yes</enabled>
    <threads>4</threads>   # parallel scan threads
  </scan_on_start>

  <directories>
    <directory>/etc/passwd</directory>   # single file
    <directory>/etc/ssh</directory>      # whole dir, no recursion
    <directory>/etc</directory>         # recursive
    <directory>/var/www</directory>     # recursive, your app code
    <directory>/tmp</directory>         # noisy - use nodiff carefully
    <directory>/bin,/sbin</directory>  # binaries, watch for tampering
  </directories>

  <ignore>
    <type>sregex</type>
    <regex>\.swp$|\.log$|~$</regex>
  </ignore>

  <nodiff>
    <path>/var/lib/apt/lists/</path>
    <path>/var/cache/</path>
  </nodiff>
</syscheck>
```

Directive	Do this
`frequency`	How often the baseline is recomputed, in seconds. 43200 = every 12 h. Lower = more sensitive, more noise.
`scan_on_start`	Build the baseline when the agent starts. Almost always yes.
`directories`	A trailing slash means "this directory only, no subdirectories".
`nodiff`	Watch these paths but do not hash them. Essential for apt and log directories, or you will drown.
`ignore`	Skip files entirely, by regex or sregex. Keep this tight.
`realtime`	Use inotify on Linux for *immediate* detection instead of waiting for the next scan.

Validate before you restart — always

```
sudo /var/ossec/bin/wazuh-logtest -t
sudo systemctl restart wazuh-manager
```

A malformed `ossec.conf` will stop the manager from starting, and if you did this remotely over SSH you will be locked out. This one command prevents that entirely.

11.4 Seeing FIM alerts

```
syscheck.type:modified
syscheck.file:/etc/passwd
syscheck.type:added
```

And to confirm monitoring is on, from the manager:

```
sudo cat /var/ossec/etc/shared/syscheck-files.txt
sudo grep "syscheck" /var/ossec/logs/ossec.log | tail -5
```

11.5 A detection exercise you can do right now

1. **Confirm `/etc/passwd` is monitored**

2. **Change something harmless**

```
sudo cp /etc/passwd /root/passwd.bak
sudo nano /etc/passwd        # add a harmless comment line, save, exit
```

3. **Watch it appear**

```
syscheck.file:/etc/passwd
```

Within 12 hours you get an alert with the change type, the before/after SHA256 hashes, and the agent name. That hash pair is forensic-grade evidence.

4. **Restore and confirm the alert goes quiet**

```
sudo cp /root/passwd.bak /etc/passwd && sudo rm /root/passwd.bak
```

12 Inventory & software inventory

Syscollector periodically reports hardware, OS, packages and running processes. It powers the vulnerability scanner and gives you a live answer to “what do we actually run?”.

12.1 What it collects

Type	Detail
Hardware	CPU, RAM, disks, network interfaces
OS	Distribution, kernel, version — feeds CVE matching
Packages	dpkg on Debian/Ubuntu, rpm on RedHat. **This is what vulnerability detection compares against.**
Applications	Installed programs, mainly for Windows
Processes	Running PIDs — useful, and off by default because it is chatty

12.2 Configure it

```
<syscollector>
  <disabled>no</disabled>
  <interval>1h</interval>
  <scan_on_start>yes</scan_on_start>
  <hardware>yes</hardware>
  <os>yes</os>
  <network>yes</network>
  <packages>yes</packages>   # needed for vulnerability detection
  <ports all="no">yes</ports>  # listens locally - NOT a scanner
  <processes>yes</processes>
</syscollector>
```

> **<ports> is not remote scanning**
>
> It only checks ports on the agent's own machine. For scanning other hosts you need nmap or Nessus — which is exactly what Volume 2 does from Kali. Do not confuse the two; the difference matters.

12.3 Reading it

Inventory in the left nav, then **Hardware**, **OS**, **Software**, **Packages**. Cross-reference a package version here against a CVE in the Vulnerability module — that is the full loop from “what is installed” to “what should I patch”.

13 Writing your own rules and decoders

This is the point where you stop being a SIEM user and start being a detection engineer. Stock Wazuh ships ~3,000 rules. Your environment will always have something specific it does not cover.

13.1 Local rules override shared rules

```
/var/ossec/etc/shared/*.xml      # stock. Never edit - lost on every upgrade.
/var/ossec/etc/local_rules.xml   # YOUR rules. Read last. Beats anything shared.
```

Only `local_rules.xml` is read from `/var/ossec/etc`. Anything you add to `shared/` is your own risk during upgrades.

13.2 Writing your first rule

Scenario: you do not want an alert every time a new user is added to the `sudo` group, but you *do* want one if it happens outside your change window.

STEP 1 **Find the fields to match on**

Do not guess. Generate a real event and read it:

```
sudo /var/ossec/bin/wazuh-logtest
```

That drops you into an interactive tester. Paste a raw log line from `/var/log/auth.log`, or press Enter for an artificial one. It tells you which rule matched and — crucially — the **decoded fields**, which is what your rule needs to reference.

```
Decoded event: 'srcip': '10.0.0.40', 'srcuser': 'ubuntu', 'dstuser': 'root',
               'id': '8A7C', 'user': 'root', 'command': '/usr/bin/passwd'
Full event: ...
**Rule 5802 matched: 'New user added to the sudo group'
```

STEP 2 Write the rule

```
<group name="lab,sudo,">

  <rule id="100100" level="10">
    <if_sid>5802</if_sid>
    <match>sudo: COMMAND = /usr/bin/passwd</match>
    <description>Password changed with sudo on $(dstuser) account</description>
    <mitre><id>T1098</id></mitre>
    <group>lab_sudo,password_change,</group>
  </rule>

</group>
```

Directive	Meaning
`id`	**Must be local**: 100000–199999. Using a stock ID silently overwrites a shipped rule.
`level`	0–15. This is the severity shown in the dashboard.
`if_sid`	Chain onto an existing rule's alerts. Cheaper and far more reliable than raw regex.
`match`	Regex against the decoded event. `|` for alternatives, `*` for wildcards.
`description`	Use `$(field)` to interpolate decoded values. This is what you read in the UI.
`group`	Tags. **Trailing comma matters** — without it, this rule's groups are appended to the parent's rather than replacing them.
`mitre`	Tactic and technique IDs, so your rule shows up in ATT&CK views.

STEP 3 Test before deploying

```
sudo /var/ossec/bin/wazuh-logtest -t   # validates ossec.conf XML
sudo /var/ossec/bin/wazuh-logtest      # paste the raw log line, watch it match 100100
```

STEP 4 Deploy and verify end to end

```
sudo systemctl restart wazuh-manager
sudo /var/ossec/bin/agent_control -R <ID>   # refresh agent config
sudo /var/ossec/bin/agent_control -i <ID>
```

Now actually generate the event on the target and search the dashboard for `rule.id:100100`. A rule that passes `wazuh-logtest` but never appears is usually missing the `agent_control -R` step.

13.3 Rule-building tactics

Technique	When to use it

`if_sid` chaining	Almost always, instead of writing raw regex. The stock decoder already did the hard parsing.
Frequency rules	Alert on a *count*: 20 failed logons in 5 minutes is interesting; one is not.
`timeframe`	Window a rule applies within.
`if_level`	Match any alert at or above a severity — handy for global escalation.
Correlation	Chain distinct rules together: authentication failure → then a successful logon from the same IP. This is how you detect a real intrusion from weak individual signals.

13.4 Frequency rule example

```
<rule id="100110" level="12">
  <if_sid>5712</if_sid>            # sshd: Authentication failed
  <same_source_ip />
  <frequency>20</frequency>
  <timeframe>300</timeframe>       # 20 failures in 5 minutes
  <description>Brute force: 20+ failed SSH logons from $(srcip) in 5 minutes</description>
  <group>authentication_failures,brute_force,</group>
</rule>
```

13.5 Suppression rules

When something is legitimately noisy, lower it rather than hiding it. Keep the evidence, lose the noise.

```
<rule id="100200" level="4">
  <if_sid>5716</if_sid>
  <same_source_ip />
  <description>Legitimate internal SSH session from lab host $(srcip)</description>
  <group>lab_noise,</group>
</rule>
```

> **Design rule of thumb**
>
> If a rule fires more than about 20 times a day and you always dismiss it, it should not be level 8. Suppress it to level 3–4 so it stays searchable. Delete it only if it is genuinely irrelevant. Never simply widen a filter to make the badge go away — that hides the alerts you have not found yet.

13.6 Dev Tools in the dashboard

Dev Tools → **Rules** lets you search every rule by ID, description, group and MITRE technique without SSH. This is the fast way to answer "does Wazuh already detect this, and what is it called?" before writing your own. **Decoders** does the same for parsers — use it to find which decoder owns a log source and which fields it produces.

13.7 Custom decoders

Only needed when you have a log source Wazuh does not parse at all — an application you wrote, an appliance, a cloud audit log. The shape:

```
<decoder name="lab-app">
  <prematch>^\s*\[LABAPP\]\s*</prematch>
</decoder>

<decoder name="lab-app">
  <parent>lab-app</parent>
  <plugin_decoder>JSON_Decoder</plugin_decoder>
</decoder>

<decoder name="lab-app-fields">
  <parent>lab-app</parent>
  <plugin_decoder>JSON_Decoder</plugin_decoder>
  <plugin_offset>12</plugin_offset>
  <path>event.action</path>
  <path>user.name</path>
  <path>src.ip</path>
</decoder>
```

Save it in `/var/ossec/etc/local_decoder.xml`, restart the manager, and check the Decoders tab in Dev Tools to confirm it loaded. Wazuh's built-in JSON decoder handles most custom formats without you writing a regex.

14 Manager command-line toolkit

The commands you will actually use, in rough order of frequency.

14.1 Health and status

```
sudo systemctl status wazuh-manager wazuh-indexer wazuh-dashboard filebeat
sudo /var/ossec/bin/wazuh-control status          # all daemons at once
sudo /var/ossec/bin/wazuh-control info            # full rule/decoder/plugin counts
```

```
curl -k -u admin:<MANAGER_PASSWORD> \
  https://127.0.0.1:9200/_cluster/health?pretty
```

Expect `"status" : "green"`. `yellow` means a replica is missing — acceptable on a single-node lab. `red` means data loss risk; check disk space first.

```
df -h /                                  # indexer is the disk hog
free -h                                    # 2 GB heap + dashboard + OS = ~4 GB needed
sudo journalctl -u wazuh-indexer -n 50 --no-pager
```

14.2 Agents

```
sudo /var/ossec/bin/manage_agents -l            # list
sudo /var/ossec/bin/manage_agents -a <NAME> -i <IP>  # add from CLI
sudo /var/ossec/bin/manage_agents -r <ID>      # remove
sudo /var/ossec/bin/manage_agents -R <ID>      # reset key
sudo /var/ossec/bin/manage_agents -e            # export the CSV
```

```
sudo /var/ossec/bin/agent_control -l            # live agents + last keep-alive
sudo /var/ossec/bin/agent_control -i <ID>      # restart
sudo /var/ossec/bin/agent_control -R <ID>      # force config re-read
sudo /var/ossec/bin/agent_control -s            # all agent status codes
```

14.3 Testing and debugging

```
sudo /var/ossec/bin/wazuh-logtest              # interactive rule tester
sudo /var/ossec/bin/wazuh-logtest -t           # validate ossec.conf
sudo /var/ossec/bin/wazuh-logtest -d           # dump decoded fields, no rule match
sudo /var/ossec/bin/wazuh-logtest -f <rulefile.xml>
```

```
sudo tail -f /var/ossec/logs/alerts/alerts.json | python3 -m json.tool
sudo tail -100 /var/ossec/logs/ossec.log
sudo grep -i "error\|denied\|refused" /var/ossec/logs/ossec.log | tail -20
```

And the one that saves the most time — confirm a rule actually fired:

```
sudo grep "rule_id\":\"100100" /var/ossec/logs/alerts/alerts.json | tail -2
```

14.4 Useful log locations

Path	Contents
/var/ossec/logs/alerts/alerts.json	Every alert. The Filebeat source.
/var/ossec/logs/ossec.log	Manager's own activity log.
/var/ossec/logs/active-response.log	Firewall blocks and other automated responses.
/var/ossec/logs/integrator.log	What was shipped to the indexer, and any failures.
/var/ossec/logs/wazuh-indexer.log	Indexer connectivity from the manager.
/var/log/filebeat/	Filebeat's own log. Read this when alerts stop appearing in the UI.
/etc/wazuh-indexer/opensearch.yml	Indexer config (you added memory_lock false here).
/var/ossec/etc/ossec.conf	Manager master config.

15 Querying Wazuh through the REST API

The API is how you do anything at scale, and how automation tools reach Wazuh. Everything below runs from inside SIEM-Ubuntu.

15.1 Get a token

```
curl -sk -X POST "https://localhost:55000/security/user/authenticate?raw=true" \
  -d '{"username":"wazuh-wui","password":"wazuh-wui"}'
# returns a long JWT. -k skips TLS verify; -s silences the progress bar.
```

```
export TOKEN="PASTE_TOKEN_HERE"
```

> **Why tokens instead of the password**
>
> Tokens expire (default 900 s, configurable). This is not a nuisance to work around — it is the pattern you will use professionally. Automation should never store an admin password.

15.2 Common calls

```
# agent list
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost:55000/agents

# one agent's details
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost:55000/agents/001

# SCA results for an agent
curl -sk -H "Authorization: Bearer $TOKEN" \
  "https://localhost:55000/sca/001?pretty"

# active response
curl -sk -H "Authorization: Bearer $TOKEN" \
  "https://localhost:55000/agents/001/config/active-response?pretty"

# Wazuh API info (shows all available endpoints)
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost:55000/?pretty
```

15.3 Querying the indexer directly

For ad-hoc forensics, the indexer gives you full Lucene power over raw alert documents.

```
curl -sk -u admin:<MANAGER_PASSWORD> \
  "https://127.0.0.1:9200/wazuh-alerts-4.x-*/_search?size=5&pretty"
```

How many alerts per agent in the last day?

```
curl -sk -u admin:<MANAGER_PASSWORD> \
  "https://127.0.0.1:9200/wazuh-alerts-4.x-*/_search?pretty" -H 'Content-Type: application/json' -d
'{
  "size": 0,
  "query": { "range": { "timestamp": { "gte": "now-24h" } } },
  "aggs": { "by_agent": { "terms": { "field": "agent.name", "size": 20 } } }
}'
```

Only critical alerts, newest first:

```
curl -sk -u admin:<MANAGER_PASSWORD> \
  "https://127.0.0.1:9200/wazuh-alerts-4.x-*/_search?pretty" -H 'Content-Type: application/json' -d
'{
  "size": 20,
  "sort": [ { "timestamp": { "order": "desc" } } ],
  "query": {
    "bool": {
      "filter": [
        { "range": { "rule.level": { "gte": 12 } } },
        { "range": { "timestamp": { "gte": "now-7d" } } }
      ]
    }
  }
}'
```

15.4 Index names you will see

Index	Contents
`wazuh-alerts-4.x-YYYY.MM.DD`	Alerts, daily index. This is what the dashboard reads.
`wazuh-states-inventory-*`	Syscollector inventory data per agent.
`wazuh-states-vulnerabilities-*`	Vulnerability scan findings.
`wazuh-archives-*`	Historic events from the archives module (disabled in your setup).

16 Troubleshooting

16.1 Dashboard will not load

Check	Expected / action
Is the VM running?	VM must be powered on, not paused. A suspended VM will not answer on any port.
Is the dashboard listening on 443?	`sudo ss -tlnp | grep :443`
Is it up but slow?	Normal on first load. Watch `systemctl is-active wazuh-indexer`.
Wrong URL?	Use `https://127.0.0.1:4433`. Plain `http` will fail — it is TLS-only.
Index unhealthy?	`curl -k ... /_cluster/health`. Red means data at risk.
Out of memory?	`free -h`. The 8 GB you allocated is close to the minimum for this stack.

16.2 Agents show "Never connected" or "Disconnected"

```
sudo /var/ossec/bin/agent_control -l              # on the manager - is it there?
ping -c2 10.0.0.30                                # from the manager, on intnet
sudo grep "Enrolled\|ERROR\|key" /var/log/wazuh-agent/wazuh-agent.log | tail -20
```

Likely cause	Fix
Wrong manager IP	Must be `10.0.0.40`, never the NAT address. Check `/var/ossec/etc/ossec.conf` on the agent.
Wrong network segment	Both agents must be on `intnet`. NAT guests cannot reach each other.
Key mismatch	Compare the agent's key with the manager's record. Reset with `manage_agents -R`.
Port 1514 blocked	Test from the agent: `timeout 5 bash -c "cat </dev/null >/dev/tcp/10.0.0.40/1514"`
Manager stopped	`systemctl status wazuh-manager` — if remoted is down nothing connects.

16.3 Alerts stop appearing in the UI, but they are in alerts.json

This is a Filebeat or indexer problem, not a manager problem. Work down:

```
sudo tail -20 /var/log/filebeat/filebeat-7.10.2
sudo systemctl status filebeat
sudo grep -i "error\|refused\|cert" /var/log/filebeat/filebeat-7.10.2 | tail
sudo tail -30 /var/ossec/logs/integrator.log
sudo systemctl status wazuh-indexer
```

A frequent cause in a TLS lab: a certificate expired or was regenerated without reloading Filebeat. Then `curl -k https://127.0.0.1:9200/_cluster/health` inside the VM is the decisive test.

16.4 SSH from Windows to Kali does not work

Known and pending from the network fix. The port forward on 2223 was created while Kali still had the clashing NAT address, and the NAT engine cached it. Kali must be powered off:

```
VBoxManage modifyvm "kali-linux-2026.2-virtualbox-amd64" --natpf2 delete ssh-host
VBoxManage modifyvm "kali-linux-2026.2-virtualbox-amd64" --natpf2 "ssh-
host,tcp,127.0.0.1,2223,10.0.3.20,22"
VBoxManage startvm "kali-linux-2026.2-virtualbox-amd64" --type headless
```

Note the explicit guest IP — that stops it chasing a moving address. Meanwhile, SSH through SIEM-Ubuntu works today:

```
ssh -p 2222 ubuntu@127.0.0.1
ssh kali@10.0.0.30
```

16.5 Configuration will not validate

```
sudo /var/ossec/bin/wazuh-logtest -t
```

Reports the file and line of the XML error. Two classic causes: unescaped `&` in a regex or description, and a `<group>` list missing its trailing comma.

> **Never restart the manager before validating**
>
> If you edit `ossec.conf` over SSH and the XML is broken, the manager will fail to start and your SSH session may survive but the agent link will not. Always `wazuh-logtest -t`, then restart. There is no reason to skip this.

16.6 Quick reference

Symptom	First thing to run
Dashboard blank	`systemctl status wazuh-dashboard`
No new alerts	`agent_control -l`, then `tail -f alerts.json`
Alerts not in UI	`tail /var/log/filebeat/filebeat-7.10.2`
Cluster red	`df -h /` and `free -h`
Manager died after an edit	`wazuh-logtest -t`
No MITRE tags	Normal — only some shipped rules carry them. Your rules need `<mitre>`.

17 Maintenance

17.1 Changing the dashboard password

```
sudo /usr/share/wazuh-indexer/plugins/opensearch-security/tools/wazuh-passwords-tool.sh \
  -a -p 'YourNewStrongPassword' -u admin
sudo systemctl restart wazuh-indexer wazuh-dashboard
```

17.2 Changing the API and internal passwords

Run `wazuh-passwords-tool.sh` without `-a` to be prompted, changing `wazuh`, `wazuh-wui`, `kibanaserver` and `admin`. Keep a copy of the output — that file is the only record of these credentials.

> **These are the credentials printed in your lab report**
>
> Change all of them before this VM goes anywhere near a network you care about. The NAT forwards bind to `127.0.0.1`, which limits exposure, but a VirtualBox NAT forward will still accept connections from anything VirtualBox considers a local peer. Assume public.

17.3 Certificates

```
sudo /var/ossec/etc/wazuh-indexer/certs/config.yml.bak   # your backup of the cert tool config
sudo ls -la /etc/wazuh-indexer/certs/
sudo openssl x509 -in /etc/wazuh-indexer/certs/indexer.pem -noout -dates
```

Regenerate all of them at once if any expire:

```
sudo /var/ossec/etc/wazuh-indexer/certs/wazuh-certs-tool.sh -A
sudo systemctl restart wazuh-indexer filebeat wazuh-manager
```

17.4 Backups worth taking

What	How
Manager config	`cp -a /var/ossec/etc /root/ossec-etc-backup-$(date +%F)/`
Indexer config + security	`cp -a /etc/wazuh-indexer/opensearch-security /root/`
Certificates	`cp -a /etc/wazuh-indexer/certs /root/certs-backup/`
VM snapshot	The real one. Power off and snapshot in VirtualBox before any config change.
Your rules	Keep `local_rules.xml` and `local_decoder.xml` in git. You will edit them often.

> **Do this right now, it takes two minutes**
>
> Power off SIEM-Ubuntu and take a VirtualBox snapshot named `wazuh-working-30sep`. You now have a known-good restore point before starting the Windows Server work in Volume 2. Every future change becomes low-risk.

17.5 Updates

Do not casually `apt upgrade` the Wazuh packages — a partial version mismatch between manager, agent and dashboard will break the pipeline in ways that are painful to debug. Upgrade the whole stack deliberately, manager last, one component at a time, and snapshot first.

18 Lab exercises to work through

Reading a guide teaches you very little. These build the actual skill, in order, each one producing evidence you can put in a portfolio.

#	Exercise	Success looks like
1	Log in, take the Overview screenshot, note your baseline alert count and agent list.	A saved image and a number you can compare against later.
2	Run three Lucene queries from section 6.2 and save each as a named search.	Three saved searches on the left rail you actually understand.
3	Pull one alert's *Full event* JSON and write down what every field means.	You could explain the alert to someone else without looking it up.
4	Complete the section 7 investigation. Follow it to a real conclusion.	Four written lines: what, when, which host, conclusion.
5	Review the CIS SCA results for Kali. Pick 5 failing checks and fix them.	Five green checks that were red, and the commands you used.
6	Add FIM on `/etc/passwd`, `/etc/ssh`, `/etc/crontab`. Do the section 11.5 exercise.	A before/after SHA256 pair in an alert.
7	Write a local rule 100100 that alerts on a specific event of your choosing. Test it with wazuh-logtest, then trigger it for real.	An alert in the dashboard with your rule ID and a MITRE tag.
8	Write a frequency rule that escalates on a pattern (failures over time, or repeated scans).	A level-12 alert that only appears when the pattern is genuinely met.
9	Add an ignore entry for a false-positive CVE. Document the justification.	One clean documented line in ossec.conf and in your notes.
10	Use the API to pull SCA results for Kali and save them as JSON.	A file proving you can automate the collection.
11	Take the snapshot from 17.4.	A restore point that exists.

19 Where to take this next

19.1 The obvious next step

Volume 2, companion to this guide: enroll your Windows Server 2022 VM as a Wazuh agent, harden it from Kali, and then run controlled simulations to prove Wazuh detects them. That takes you from "I have a SIEM" to "I have a validated detection capability" — a much more valuable claim, and a far better portfolio artefact.

19.2 After that

Direction	Why it is worth doing
Suricata on SIEM-Ubuntu	Wazuh sees what hosts *report*; Suricata sees what actually crosses the wire. Install it on Ubuntu, ship its `eve.json` into Wazuh via a Filebeat input, and you now have real network detection. This is the single highest-value addition to your lab.
Windows Client + Active Directory	Enroll your Windows 11 VM, then promote WinServer to a domain controller. Kerberos, group policy, logon types and domain events are where most real-world Windows security happens.
osquery	SQL-query your endpoints like a database. Pairs well with Wazuh as a second opinion.
Sigma rules	Portable detection-rule format. Write one, run it in Wazuh, then you have a transferable skill rather than a vendor-specific one.
Sysmon on Windows	Far richer process/network telemetry than stock Windows event channels. Covered in Volume 2.
A second Kali/attacker VM	Separates attacker and analyst. Right now Kali is both your attack tool and a monitored agent — which is actually useful, because you can watch your own actions appear in the SIEM.
Write it up	A GitHub repo with your `local_rules.xml`, your FIM config, your SCA before/after, and a README explaining each. This is what gets you a job, far more than a certificate does.
TryHackMe / PortSwigger	Structured practice on attack techniques, to understand what you are detecting. You have a Security+ certificate; these build the applied half.

19.3 A note on where you actually are

You have, in about a day: a working three-node SIEM with TLS, a custom Filebeat pipeline, a live agent fleet, and — more importantly — you debugged a multi-layer network fault end to end without panicking.

Most people who take a Security+ course never get a working SIEM. The thing that got you through yesterday was not memorising commands, it was forming a hypothesis, testing it against evidence, and forming a better one. That is the actual job. Everything in this document is just tooling for doing it systematically.

Document conventions

Every command, IP, port and path in this guide was verified against your actual VMs on 30 September 2026. Values marked *planned*, *optional* or *if different* are the only ones you may need to adjust. If you change your lab topology, this document drifts — keep it in the same place as your config backups so you know how old it is.